

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030027

Name: Venkata Sai Krishna

SEC-8

SKILL-4:

Aim: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

PROCEDURE :

1. Command Tokenization and Whitespace Handling :

```
venkatsaikrishna5@VSK-HP:~$ mkdir skill4
venkatsaikrishna5@VSK-HP:~$ cd skill4
venkatsaikrishna5@VSK-HP:~/skill4$ nano tokenize.c
venkatsaikrishna5@VSK-HP:~/skill4$ gcc tokenize.c -o tokenize
venkatsaikrishna5@VSK-HP:~/skill4$ ./tokenize
Enter a command: ls -l /home

Tokens:
Token: [ls]
Token: [-l]
Token: [/home]
venkatsaikrishna5@VSK-HP:~/skill4$ ./tokenize
Enter a command: gcc file.c -o file

Tokens:
Token: [gcc]
Token: [file.c]
Token: [-o]
Token: [file]
```

2. Empty input :

```
venkatsaikrishna5@VSK-HP:~/skill4$ ./tokenize
Enter a command:
Empty input.
```

3. Command Parsing and Parse Tree Generation :

```
venkatsaikrishna5@VSK-HP:~/skill4$ nano parser.c
venkatsaikrishna5@VSK-HP:~/skill4$ gcc parser.c -o parser
venkatsaikrishna5@VSK-HP:~/skill4$ ./parser
Enter command: ls -l /home

Parse Tree:
COMMAND: ls
|
+-- ARGUMENT: -l
|
+-- ARGUMENT: /home

Syntax validation successful.
Execution structure generated.
venkatsaikrishna5@VSK-HP:~/skill4$ ./parser
Enter command: gcc file.c -o file

Parse Tree:
COMMAND: gcc
|
+-- ARGUMENT: file.c
|
+-- ARGUMENT: -o
|
+-- ARGUMENT: file

Syntax validation successful.
Execution structure generated.
```

4. Empty command/error:

```
venkatsaikrishna5@VSK-HP:~/skill4$ ./parser
Enter command:
Syntax Error: Empty command.
```

Code:

```
GNU nano 8.7.1                               tokenize.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    char *token;

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty input.\n");
        return 0;
    }

    printf("\nTokens:\n");

    token = strtok(input, " \t");

    while (token != NULL)
    {
        printf("Token: [%s]\n", token);
        token = strtok(NULL, " \t");
    }

    return 0;
}
```

```

GNU nano 8.7.1                               parser.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) == 0)
    {
        printf("Syntax Error: Empty command.\n");
        return 0;
    }
    printf("\nParse Tree:\n");
    char *command = strtok(input, " ");
    if (command == NULL)
    {
        printf("Syntax Error.\n");
        return 0;
    }
    printf("COMMAND: %s\n", command);
    char *argument = strtok(NULL, " ");
    while (argument != NULL)
    {
        printf("  |\n");
        printf("  +-- ARGUMENT: %s\n", argument);
        argument = strtok(NULL, " ");
    }
    printf("\nSyntax validation successful.\n");
    printf("Execution structure generated.\n");
    return 0;
}

```

Observation :

1. The input command was successfully split into individual tokens by identifying spaces and tabs as delimiters.
2. Commands and arguments were correctly identified and represented as a simple parse tree.
3. Extra whitespace and empty inputs were handled correctly.
4. Syntax was validated, and errors such as empty commands were detected.
5. The parsed tokens were used to generate a basic execution structure.

Result :

The program successfully performed **command tokenization and parsing**, including token identification, whitespace handling, syntax validation, parse tree generation, error detection, and execution structure creation.